

ISART DIGITAL

AUTONOMOUS AI RACING DRONE FLIGHT

Antoine MORDANT
Lenny RABINNE
Matias HENO

Game Programming
AI Specialization

October 9, 2025 - June 2, 2026

ISART DIGITAL
Supervisors:
Maël ADDOUM

Aurélien LHERBIER

Industry Supervisor:
Armand POUYA

Abstract

The development of an autonomous AI capable of piloting high-speed FPV (First Person View) drones within complex, realistic environments is presented. Using Deep Reinforcement Learning (DRL) and the Proximal Policy Optimization (PPO) algorithm, an agent was trained to navigate within the TRYP FPV simulator. This project, conducted in collaboration with Undead Battery, leverages ultra-realistic physics to bridge the gap between simplified simulations and real-world flight dynamics. The final implementation aims to provide a generalized navigation model adaptable to unknown environments and various drone configurations.

Résumé :

Ce projet présente le développement d'une IA autonome capable de piloter des drones FPV (First Person View) à haute vitesse dans des environnements complexes et réalistes. En utilisant l'apprentissage par renforcement profond (DRL) et l'algorithme Proximal Policy Optimization (PPO), un agent a été entraîné à naviguer au sein du simulateur TRYP FPV. Ce travail, réalisé en collaboration avec Undead Battery, exploite une physique ultra-réaliste pour réduire l'écart entre les simulations simplifiées et la dynamique de vol réelle. L'implémentation finale vise à fournir un modèle de navigation généralisé, adaptable à des environnements inconnus et à diverses configurations de drones.

Contents

1 Undead Battery **4**

2 Introduction **4**
2.1 Drone . . . 4
2.1.1 The Pitch Axis (Blue) . . . 5
2.1.2 The Roll Axis (Green) . . . 5
2.1.3 The Yaw Axis (Red) . . . 5
2.1.4 Throttle and the Action Space . . . 5
2.2 Motivation . . . 6
2.3 State of the Art . . . 6
2.4 Objectives and Organization . . . 7

3 Methodology **8**
3.1 Algorithmic Approach . . . 8
3.1.1 Observations . . . 8
3.1.2 Actions . . . 9
3.1.3 Rewards . . . 9
3.2 Development Workflow . . . 9
3.2.1 Preliminary tests . . . 9
3.2.2 Spline-driven navigation . . . 10
3.2.3 Target-driven navigation . . . 11
3.2.4 Gate-driven navigation . . . 13
3.2.5 High-Fidelity Integration . . . 15
3.2.6 Track-based navigation . . . 16
3.2.7 Advanced Track-based navigation . . . 18
3.3 Technical Stack . . . 20

4 Results **20**
4.1 Simplified simulation results . . . 20
4.1.1 Target based . . . 21
4.1.2 Target based - Quantitative Analysis . . . 21
4.1.3 Target based - Qualitative Analysis . . . 22
4.1.4 Gate based . . . 22
4.2 High fidelity results . . . 22
4.2.1 Track based . . . 23
4.2.2 Track based - Quantitative Analysis . . . 23
4.2.3 Track based - Qualitative Analysis . . . 24
4.2.4 Advanced track based . . . 25
4.2.5 Advanced track based - Quantitative Analysis . . . 25
4.2.6 Advanced track based - Qualitative Analysis . . . 25
4.3 Statistic comparison . . . 26

4.4 Limitations 27

5 Conclusions and Future Perspectives **28**

1 Undead Battery

Undead Battery [1] was founded by Armand Pouya and is mainly known for the development of TRYP FPV, a high-quality drone racing simulator available on Steam [2]. The studio is specialized in creating very realistic flight physics and high-end graphics using Unreal Engine 5. Since its release, TRYP FPV has been recognized as one of the most accurate simulators in the FPV community.

This research project is part of a plan by Undead Battery to improve the player experience. The goal is to add advanced AI opponents that can fly like real pilots, making the game more immersive and challenging. To help with this research, the TRYP FPV platform was provided by the company to be used as a professional testing environment.

Using realistic physics and the specific drone models created by the studio, the reinforcement learning agent is trained in conditions that are as close to reality as possible. The simulator provides a complex environment where gravity, wind, and momentum are accurately calculated. Consequently, the gap between simple computer simulations and real-world flight is reduced. This collaboration allows the team to test the AI on a professional tool, ensuring that the results are relevant for the final game and for the drone industry in general.

2 Introduction

The challenge of autonomous high-speed navigation in a 3-dimensional environment remains a critical frontier in both robotics and competitive gaming. The primary objective of this research is to create a neural network capable of piloting an FPV drone in any environment with realistic physics.

2.1 Drone

To develop an effective AI pilot, the mechanical principles of a quadcopter must be established. As shown in Figure 1, drone flight is controlled by managing its orientation along three main axes. These rotations are achieved by varying the speed of the four motors.



Figure 1: Yaw, pitch and roll rotations of a Quadcopter

2.1.1 The Pitch Axis (Blue)

The lateral axis, represented by the blue arrow, is used for Pitch movements. Rotation around this axis allows the drone to tilt its nose upward or downward. In a quadcopter, this movement is created by changing the speed of the front motors relative to the back motors. When the drone tilts forward, a portion of the total thrust is directed backward, which pushes the drone forward.

2.1.2 The Roll Axis (Green)

The longitudinal axis, indicated by the green arrow, controls the Roll of the drone. By rotating around this axis, the drone tilts to the left or to the right. This is achieved by increasing the speed of the motors on one side while decreasing the motors on the opposite side. This movement is necessary for cornering and lateral adjustments during high-speed racing.

2.1.3 The Yaw Axis (Red)

The vertical axis, shown by the red arrow, is responsible for Yaw. Rotation around this axis allows the drone to change its heading, meaning turning either left or right, without tilting. This movement is managed by the torque created by the propellers. The diagonally opposite motors spin in the same direction; by increasing the speed of one pair while decreasing the other, the drone is forced to rotate around its center.

2.1.4 Throttle and the Action Space

In addition to these three axes whose underlying actions are related to distributing the thrust among the 4 motors, the Throttle controls the total thrust produced by all four motors simultaneously. By increasing the throttle, more lift is generated, and the drone moves upward.

In the context of reinforcement learning within TRYP FPV, these four parameters (Pitch, Roll, Yaw, and Throttle) constitute the Action Space of the neural

network. Every few milliseconds, the AI agent must decide the optimal value for each of these four inputs to navigate the environment and avoid obstacles.

2.2 Motivation

While traditional AI often relies on scripted paths or simplified physics, the complexity of FPV flight-characterized by four degrees of freedom and high velocity-demands a more robust approach. Deep Reinforcement Learning was selected to handle the vast parameter space required for credible flight behavior. This project explores a domain rarely addressed in commercial video games: the intersection of competitive drone racing and generalized autonomous navigation.

2.3 State of the Art

The use of autonomous drones has significantly increased in recent years and now extends across many different fields. In logistics, companies such as Amazon Prime Air [3] are developing drone delivery systems to transport packages autonomously. In the entertainment industry, drone swarms are increasingly used for large-scale light shows as an alternative to traditional fireworks, offering highly synchronized and reconfigurable visual performances. Drones are also becoming increasingly used in emergency and rescue operations, where they can assist with aerial reconnaissance and situation assessment, and in some cases support more advanced intervention tasks depending on the scenario. At the same time, drones have become increasingly present in military applications, particularly through the rise of drone warfare observed in recent conflicts. These examples highlight the growing importance of autonomous navigation, obstacle avoidance, and adaptation capabilities in drone systems.



Figure 2: Amazon's Prime Air Delivery Drone



Figure 3: 5,000 Drone Christmas Light Show, Sky Elements, 2024

The field of autonomous drones has been studied by several researchers. A famous example is the work done by Kaufmann and al. in the article "Champion-level drone racing using deep reinforcement learning" [4]. In this study, an AI was trained to beat human world champions on a specific, real-world race track. However, these existing solutions are often limited because they are designed for one specific environment. In our project, the goal is to create an agent that can adapt to unknown environments and avoid obstacles, rather than simply repeating a single race track. Furthermore, many existing AI drone projects are limited to 2D or simplified 3D physics [5] [6]. By using TRYP FPV, these limitations are avoided, as the AI must learn to handle professional-grade flight dynamics.

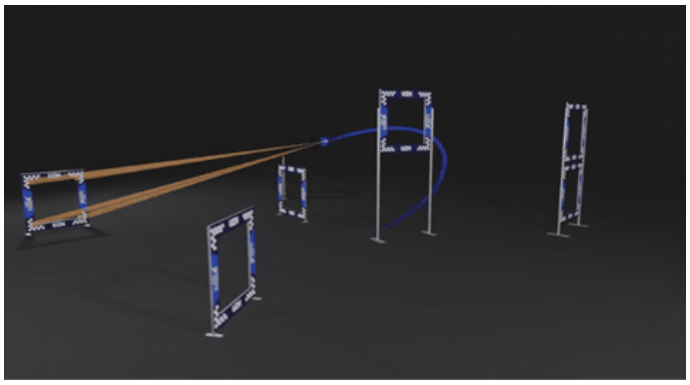


Figure 4: Champion-level Drone Racing using Deep Reinforcement Learning

2.4 Objectives and Organization

To address these challenges, the project is structured into several stages. First, the fundamental principles of reinforcement learning were tested using the Learning Agents plugin [7] in Unreal Engine 5 with cars, and hence 2D navigation. Next, custom simplified drone models were developed to evaluate AI's ability to navigate

in a 3D environment. Finally, the system was integrated into TRYP FPV to achieve a higher level of realism.

The main objective of the drone's final behavior is twofold. The first priority is to ensure that it learns precise and reliable navigation trajectories, allowing it to avoid obstacles efficiently and consistently reach its target. In addition, the agent must be able to generalize across multiple race layouts rather than memorize a single track, ensuring adaptability to different course configurations. Once this navigation objective is achieved, the second priority becomes speed optimization, enabling the drone to complete the course as quickly as possible, since the task remains fundamentally a race.

The focus of the research is on the reward model, which defines how the AI is evaluated during training by indicating whether its behavior is optimal or not. This part of the work is based on recent research on reward design in deep learning [8].

The rest of this paper is organized as follows: The third section describes the technical methodology and the tools used for training. The fourth section presents the results obtained with the simulator and the final section discusses the limitations and future improvements of the system.

3 Methodology

A step-by-step approach was chosen to ensure that the AI could learn basic movements before trying to solve complex racing tasks.

3.1 Algorithmic Approach

The Proximal Policy Optimization (PPO) algorithm was selected as the core learning mechanism for this project. This choice was motivated by the algorithm's reputation for stability and its balance between ease of tuning and sample efficiency. By using PPO, the risk of making large and destructive updates to the neural network is minimized. This is essential when dealing with the sensitive and fast-paced physics of an FPV drone. The neural network architecture is managed and executed through a dedicated Learning Agent Manager, which acts as the bridge between the AI model and the Unreal Engine environment.

The training process is structured around a continuous reinforcement learning loop. To understand how the agent learns to fly, three fundamental concepts must be defined: Observations, Actions, and Rewards.

3.1.1 Observations

The observations constitute the "sensory input" of the AI agent. They represent the data collected from the simulation at each time step to inform the neural network about its current state. These inputs include both internal data (such as the

drone's velocity and orientation) and external data (such as the distance to the next objective or obstacle detection). Without accurate observations, the agent would be unable to perceive the consequences of its movements or the layout of its environment.

3.1.2 Actions

The actions are the outputs generated by the neural network in response to the observations. In this project, the actions correspond to the flight control signals that are sent to the drone's motors. These outputs are translated into physical movements along the axes of Pitch, Roll, Yaw, and Throttle. The goal of the learning process is for the agent to discover which combination of actions leads to the most efficient and safest flight path.

3.1.3 Rewards

The reward system is the primary feedback mechanism used to guide the agent's behavior. It is a mathematical function that assigns a score to the agent's performance at every step. Positive rewards are granted for desirable behaviors, such as maintaining high speed or following a target, while negative rewards are applied for failures, such as colliding with obstacles. Through millions of iterations, the agent's objective is to maximize the total cumulative reward, which effectively teaches it to fly optimally.

3.2 Development Workflow

The methodology is divided into three main phases:
Initial Testing: The Learning Agents plugin was first used with a simple car tutorial. This allowed the team to understand how observations and actions are handled by the software.
Prototype Drones: A basic drone was built with simple physics to test if the agent could follow a 3D path (a spline) and then reach randomly placed targets. During this phase, the reward system was optimized to encourage both speed and precision.
Final Integration: The AI was moved to the TRYP FPV environment. A specific plugin was developed so that the autonomous flight technology could be used across multiple projects.

3.2.1 Preliminary tests

The initial phase of this project aimed at familiarizing with the Learning Agents plugin in Unreal Engine 5 [9]. The objective was to understand how to properly set up a reinforcement learning workflow, including the definition of observations,

actions, and reward functions, as well as managing training sessions, saving trained models, and resuming training when needed.

To achieve this, the official documentation and tutorial provided by Epic Games [9] were followed, offering a structured introduction to the plugin's architecture and usage.

As a first practical experiment, a simplified scenario was implemented in which a vehicle is trained to follow a predefined path represented by a spline. A spline can be understood as a smooth, continuous curve defined by a set of control points, commonly used in game development to represent trajectories.

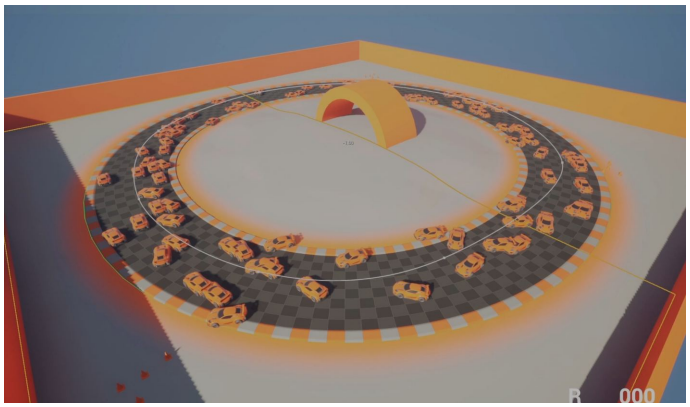


Figure 5: Spline-driven Car Training

In this setup, the vehicle evolves on a flat surface, reducing the problem to a two-dimensional navigation task. This simplified environment allows for faster training iterations and easier debugging, making it particularly suitable for understanding the core mechanisms of reinforcement learning within the plugin before moving to more complex three-dimensional drone dynamics.

3.2.2 Spline-driven navigation

In this phase, the vehicle-based setup was replaced by a simplified drone model in order to introduce three-dimensional navigation. The drone physics were intentionally simplified: instead of simulating individual propellers, a single vertical force was applied, along with approximated physical calculations.

Despite these simplifications, the control inputs remained consistent with real FPV drone commands, namely throttle, pitch, roll, and yaw. This design choice ensures that the learned behavior remains transferable to more realistic flight models.

The agent's objective was to follow a predefined spline, now extended into a three-dimensional space. Unlike the previous 2D setup, the drone must maintain its position along the trajectory while controlling altitude and orientation. The reward function was designed to encourage the agent to stay close to the spline and avoid collisions with the ground, thereby improving trajectory precision.

In addition, initial experiments were conducted on the reward system to encourage higher speeds. The goal at this stage was not to fully optimize performance, but rather to start exploring how to guide the agent toward faster behaviors, in line with the racing objective of the project. These early tests helped identify how speed-related incentives could be integrated without immediately compromising flight stability.

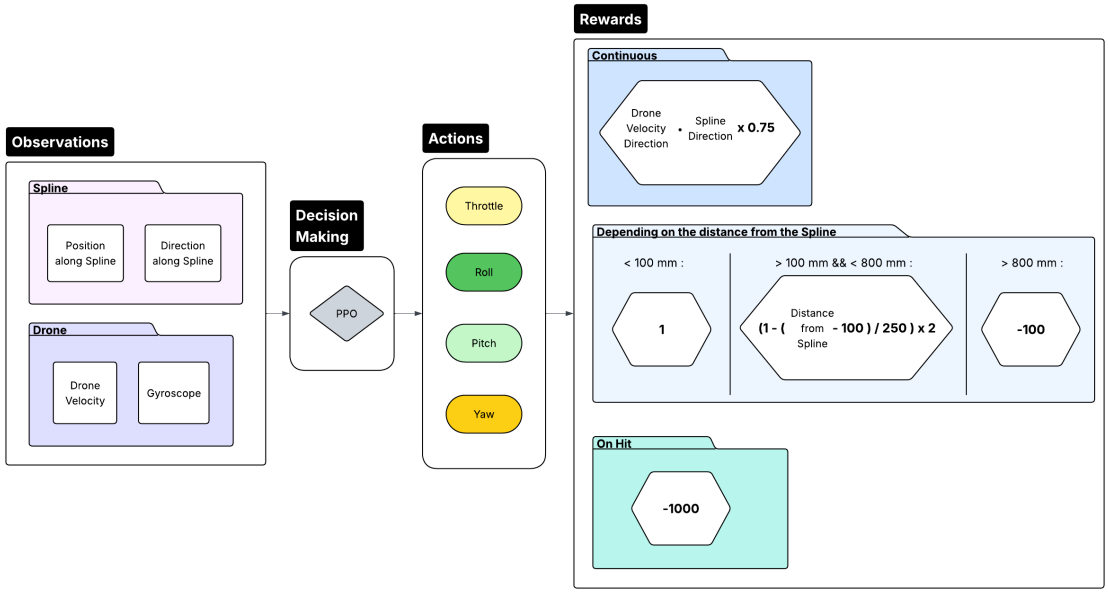


Figure 6: Spline-driven Training Overview

3.2.3 Target-driven navigation

In this phase, the spline-based guidance was removed and replaced with a dynamic target represented by a sphere. Instead of following a predefined path, the agent is only provided with the position of the target and must learn how to reach it by determining its own trajectory.

The training takes place inside a large cubic environment defining the boundaries of the map. If the drone collides with these boundaries, it receives a penalty and the episode is reset. This constraint introduces basic spatial limits while remaining simple enough for early-stage learning.

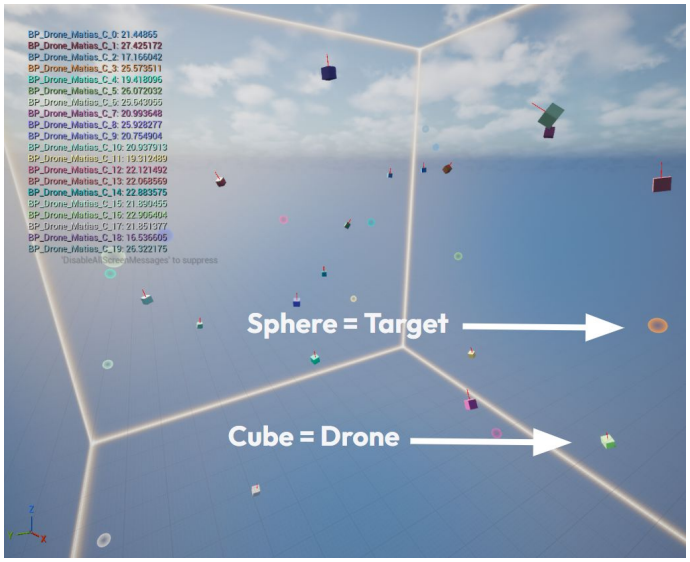


Figure 7: Target-driven Training

To encourage adaptability and generalization, both the drone's initial position and the target location are randomized at the beginning of each episode. Additionally, once the target is reached, it is immediately repositioned to a new random location, forcing the agent to continuously adapt its behavior in a changing environment.

To accelerate training observation and monitoring, approximately twenty drones are trained simultaneously within the same environment, each identified by a distinct color. These agents do not interact with each other: collisions between drones are disabled, and each drone can only reach its own target. This setup is purely designed to facilitate supervision and visualization of the training process.

The drone perceives its surroundings using a set of raycasts, allowing it to detect nearby obstacles and avoid collisions. The reward system is primarily designed to encourage reaching the target while penalizing collisions with the environment.

At this stage, the focus is not on speed but on precision and robustness. The objective is to ensure that the agent can reliably reach targets in various configurations before introducing additional constraints related to racing performance. Speed optimization is therefore intentionally deferred to later stages of the project.

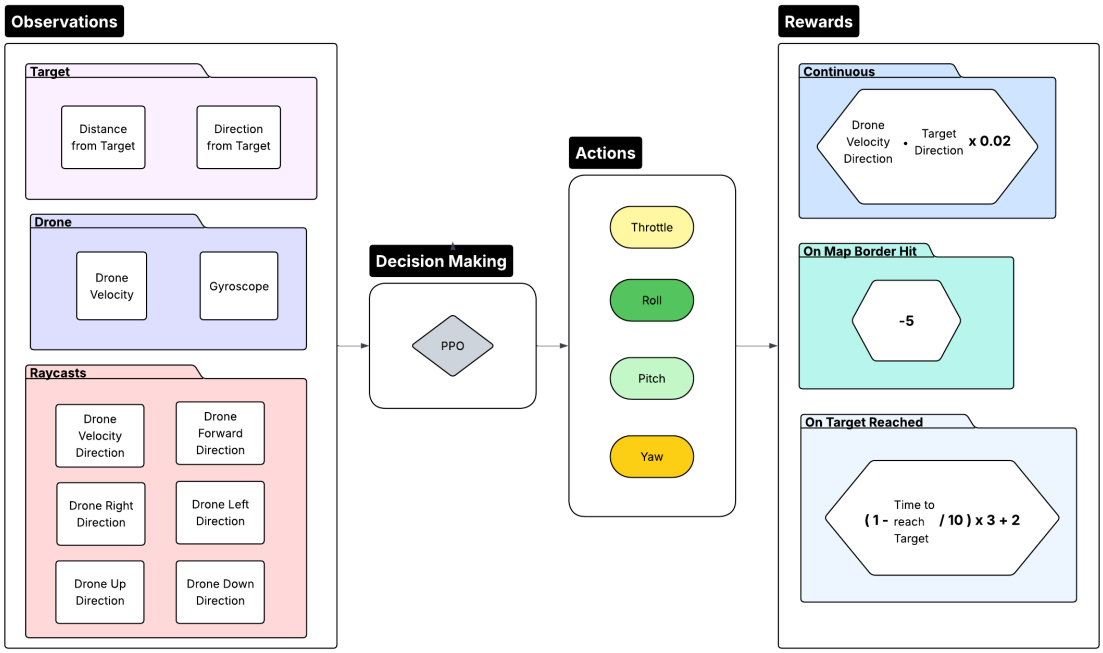


Figure 8: Target-driven Training Overview

3.2.4 Gate-driven navigation

This phase extends the environment-driven setup by replacing the spherical target with a gate. At any given time, only a single gate is active in the environment. Once the gate is successfully validated, it is repositioned to a new random location, maintaining the same dynamic and adaptive training structure as in the previous phase.

The gates are represented as circular or rectangular rings of varying sizes that the drone must pass through. A gate is only validated when crossed in the correct direction, which is determined by the drone's velocity vector rather than its orientation. This introduces a more realistic constraint, requiring precise control over the drone's movement.

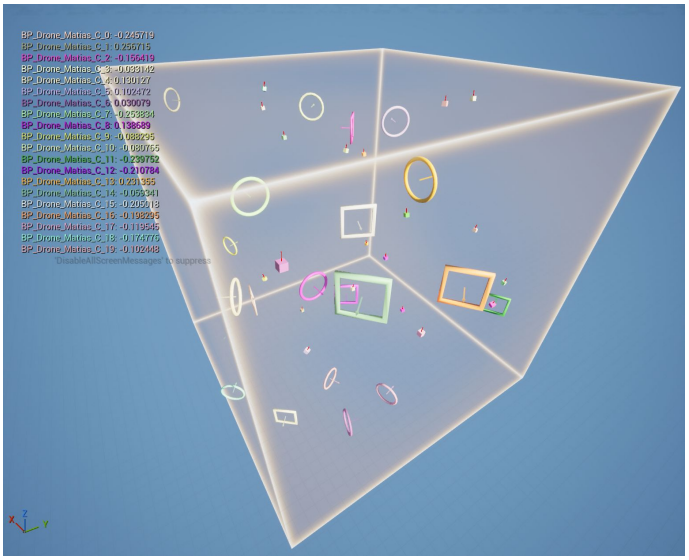


Figure 9: Gate-driven Training

Unlike the previous setup, the frame of the gate is solid, meaning that collisions are now possible. This makes the gate both a target and an obstacle, forcing the agent to carefully adjust its trajectory to avoid crashing while still achieving its objective.

To improve obstacle perception, additional raycasts were introduced in front of the drone, forming a cone-shaped detection pattern. The cone is oriented along a direction between the forward and upward vectors (approximately 45 degrees from forward), better matching the drone's motion when flying at high speed.

This phase reinforces the agent's ability to navigate through constrained spaces while maintaining control accuracy, bringing training conditions closer to real racing scenarios.

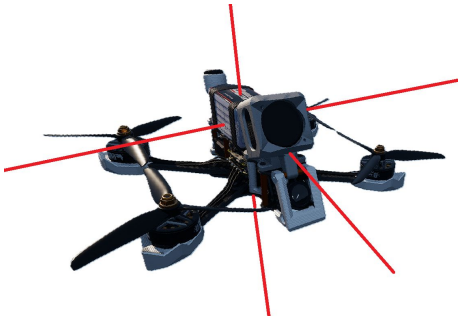


Figure 10: Repartition of the agent's raycasts

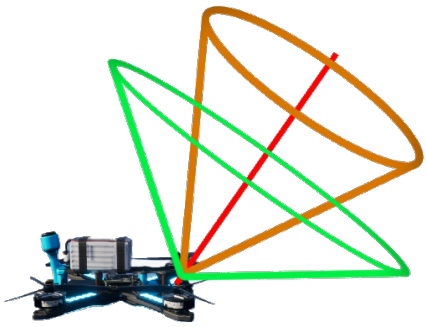


Figure 11: Agent's Cone Observations

3.2.5 High-Fidelity Integration

Up to this point, all experiments were conducted in a separate, simplified environment designed to progressively develop and validate the training pipeline. The next step consists of transferring this system into the TRYP FPV project in order to take advantage of its high-fidelity physics, which closely replicates real-world drone dynamics.

To ensure a clean and modular integration, a custom Unreal Engine plugin was developed, encapsulating the entire reinforcement learning framework. This approach allows the training system to be easily integrated into the TRYP FPV project without affecting its existing architecture.

An initial minimal training setup was implemented to validate the correct integration of the system within the simulator. This step ensured that observations, actions, rewards, and training processes were working as expected under the new physics conditions.

Once validated, the most advanced training configuration, the gate-driven navigation system, was integrated into the TRYP FPV environment.

This transition represents a critical step in the project, as it bridges the gap between simplified simulations and realistic drone behavior, bringing the agent closer to real-world applicability.

3.2.6 Track-based navigation

In this final phase, the training setup evolves from single-target navigation to a full racing scenario. Instead of interacting with one gate at a time, the drone must navigate through a sequence of gates forming a complete track.

The gates are still randomly positioned to preserve the generalization capability developed in previous stages. The agent must therefore learn not only to reach individual targets, but also to chain multiple navigation decisions in a coherent trajectory.

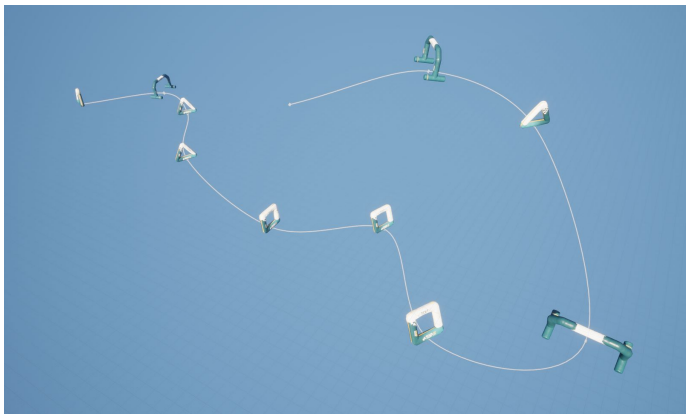


Figure 12: Target-driven Training

This introduces a higher level of complexity as the drone must anticipate upcoming gates while maintaining stable and efficient motion. The task shifts from reactive navigation to more structured and continuous trajectory control.

To further increase the difficulty and realism of the environment, obstacles are progressively introduced along the track. These elements force the agent to adapt its trajectory and find viable paths while maintaining progress toward the next gate.

This phase represents the closest approximation to a real racing scenario, which combines precision, adaptability, and trajectory optimization in a dynamic environment.

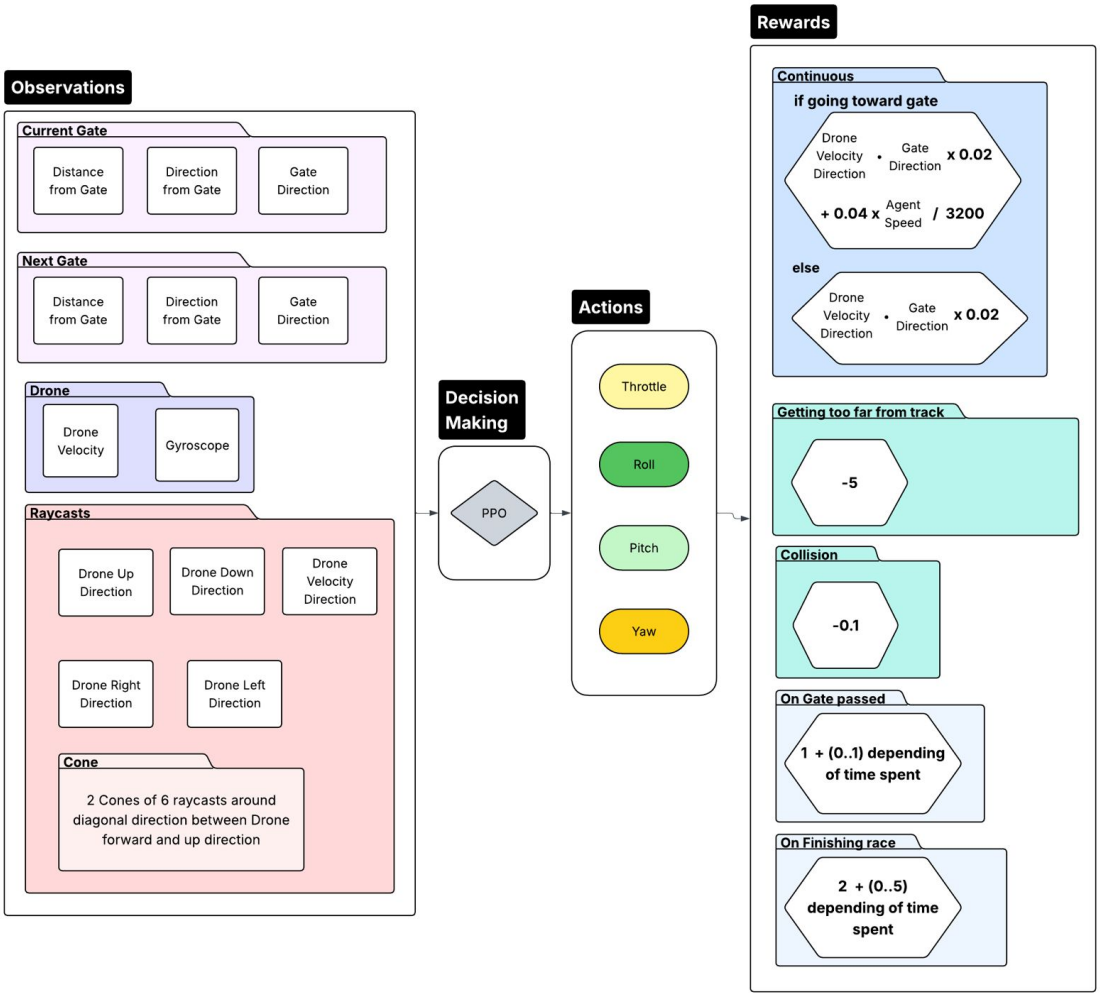


Figure 13: Track-driven Training Overview

3.2.7 Advanced Track-based navigation

In this final phase, obstacles are introduced along the track to further increase the difficulty and realism of the environment. These elements force the agent to adapt its trajectory and find viable paths while maintaining progress toward the next gate.



Figure 14: Target-driven Training

This phase represents the closest approximation to a real racing scenario, combining precision, adaptability, and trajectory optimization in a dynamic environment.

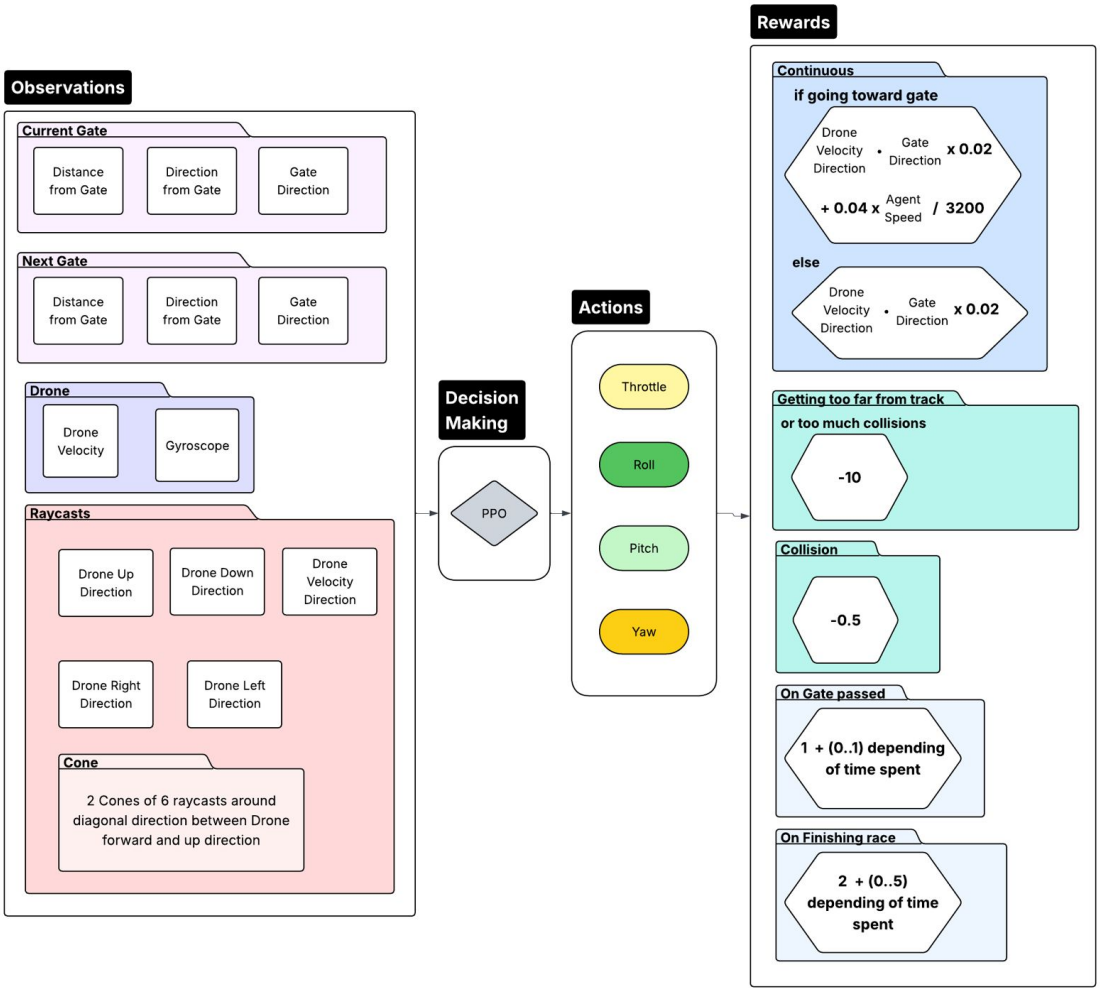


Figure 15: Advanced Track-driven Training Overview

3.3 Technical Stack

- **Engine**: Unreal Engine 5 (version 5.7).
- **AI Framework**: Learning Agents Plugin [9] (C++ and Blueprints).
- **Monitoring**: Tensorboard for learning curve visualization.
- **Versionning**: GitHub/Perforce
- **Bibliography**: Zotero

4 Results

This project yielded a significant amount of data across its various phases. This section breaks down the findings phase by phase, combining qualitative observations with quantitative metrics from TensorBoard. Our analysis tracks four key indicators:

- **Average Episode Length:** The average duration (or number of steps) an agent survives before a reset occurs (e.g., due to a collision).
- **Average Reward:** The mean reward the agent receives per tick during a training iteration.
- **Average Return:** The total accumulated reward over an entire episode, which serves as the primary measure of a run's success.
- **Average Reward Sum:** The total sum of rewards collected across a specific training batch, used to monitor overall learning progress.

4.1 Simplified simulation results

Prior to demonstrating the integration of our model within the complex flight dynamics of the TRYP FPV simulator, this section presents the intermediate results obtained on our simplified evaluation framework. This initial experimental phase aimed to empirically validate the convergence of the Proximal Policy Optimization (PPO) algorithm as well as the mathematical consistency of our reward functions. The key metrics derived from TensorBoard, presented below, analyze the joint evolution of the agent's stability (mean episode length) and the efficiency of its policy (cumulative and mean rewards accumulated at each iteration).

4.1.1 Target based

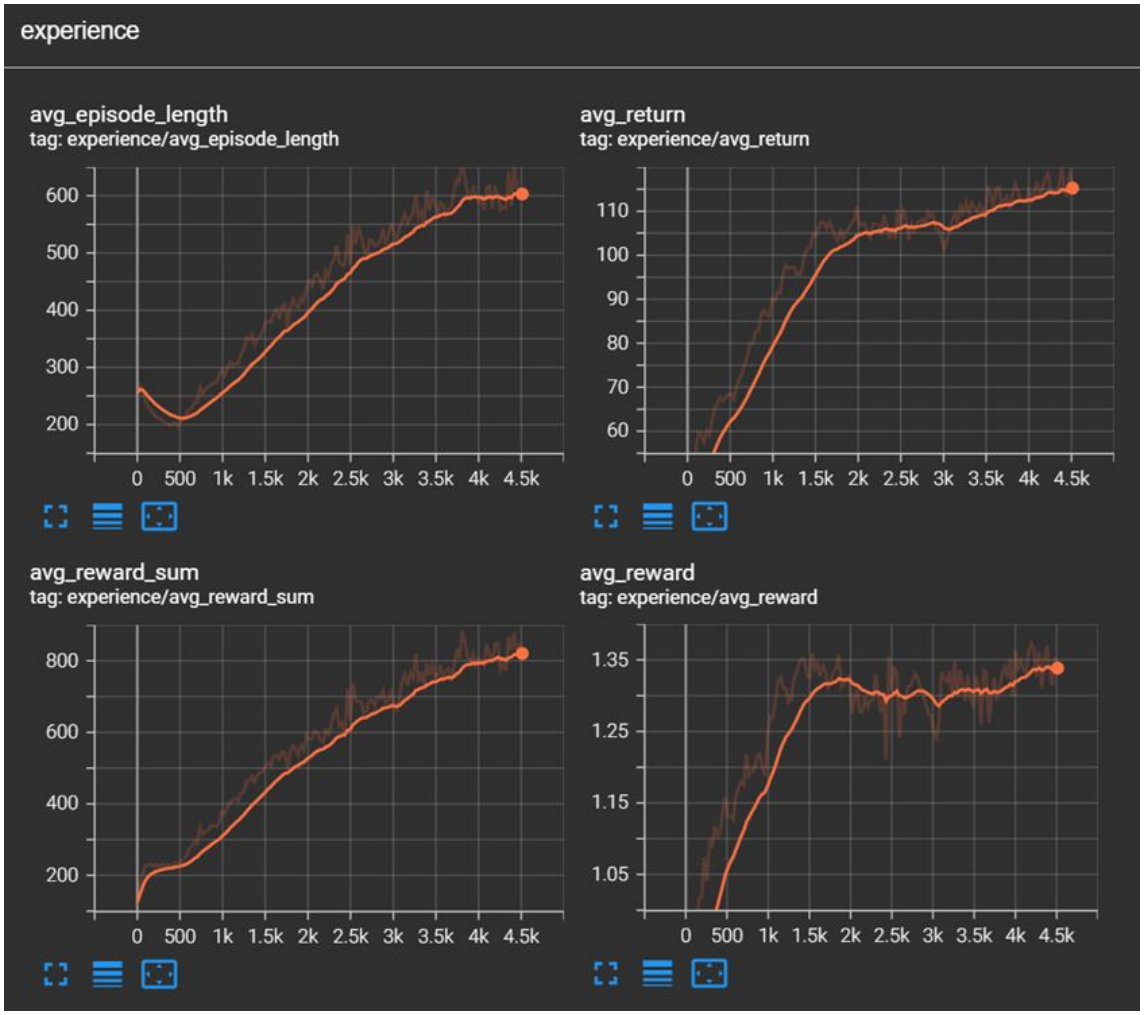


Figure 16: Simplified Simulation - Target Based Training Graphs

4.1.2 Target based - Quantitative Analysis

Every metric, as it can be seen in Figure 16, shows an overall increase in every category. However, the average reward seems to stagnate after 2 000 steps. It indicates that the agents found optimized movement methods so the average reward stopped increasing. At the same time, the average episode length kept increasing, showing that the agent learned how to avoid ending an episode manually, which, as consequences, led to an increase of the average reward sum, since the reward is maximized and the episodes are longer.

4.1.3 Target based - Qualitative Analysis

While the agent demonstrates high proficiency in open spaces, "wobbling" behaviors are observed during tight turns. This suggests a need for further refinement of the physics-stepping frequency or the reward penalty for excessive angular velocity. Integration into TRYP FPV has successfully validated that the model can handle high-fidelity drag and lift forces.

4.1.4 Gate based

This phase did not yield any major results, as its main objective was to test gate collision boundaries and fine-tune the drone's sensor cone mentioned earlier. We aimed to find the right balance between detection accuracy and observation complexity. A higher number of raycasts provides more precise obstacle detection, but it also increases the number of observations, making the training process longer and harder for the agent to compute.

After several tests, we found the optimal balance: a 45-degree forward-and-upward tilt to match the drone's orientation at high speeds, and a configuration of 13 raycasts arranged as two concentric hexagons with a single raycast in the center. Once this setup was validated, we quickly moved to the next phase, as our priority was to deploy the model on TRYP FPV with realistic drone physics as soon as possible.

4.2 High fidelity results

Following the initial validation phase in the simplified environment, the pipeline was deployed within the high-fidelity TRYP FPV simulator under Unreal Engine 5. This section evaluates the agent's ability to generalize its learned flight policy when subjected to ultra-realistic aerodynamics, complex physics, and demanding track layouts.

4.2.1 Track based



Figure 17: High Fidelity Simulation - Track Based Training Graphs

4.2.2 Track based - Quantitative Analysis

As shown in Figure 17, training the drone on the complete race track was more difficult than in the previous experiments. During the first training session, the agent initially showed good progress, with all metrics increasing steadily. However, this progress slowed down over time. Around 6,000 steps, the average episode length stagnated at around 300 steps, meaning that the drone struggled to survive longer runs. Around 10,000 steps, the average reward also stopped improving and stayed close to -0.03. During this phase, the drone had a very specific behavior: it was able to reach the first gate at high speed, but then struggled to quickly reorient itself toward the next gate. It only succeeded when the first two gates were relatively aligned. When a sharper turn was required, the drone could not rotate fast enough and often left the race boundaries.

To continue the learning process, additional training sessions were performed while keeping the previously learned model. The second training session, lasting around 15,000 steps, showed almost no improvement and most metrics remained stable. The breakthrough happened during the third training session, where the agent found a strategy to solve this problem. The drone progressively improved its ability to turn toward the next gate, increasing its record of gates passed per run until it was eventually able to complete the entire race. This resulted in a strong increase in all metrics, especially average episode length and average reward sum. Finally, a fourth training session of around 10,000 steps was used as a fine-tuning phase. Improvements were smaller during this stage, but it helped increase the success rate of completed races and further stabilize the learned behavior.

4.2.3 Track based - Qualitative Analysis

Qualitatively, the trained drone demonstrates good performance on the racing task and is able to complete the course successfully in most attempts, with an estimated success rate of around 90 percent. The agent pilots the drone accurately while maintaining relatively high speed, resulting in smooth navigation between gates and reliable completion of the race. Overall, the learned policy produces stable and consistent behavior.

However, some limitations remain visible during flight. Although the drone generally follows efficient paths, its trajectory is not always fully optimized. In particular, when the next gate is still far away, the drone occasionally performs unnecessary yaw rotations while continuing along the same direction of movement. These rotations have little impact on performance but reveal remaining inefficiencies in the learned control policy. Additionally, the drone still occasionally collides with the edges of gates, indicating that trajectory anticipation and precision could still be improved in some situations.

4.2.4 Advanced track based

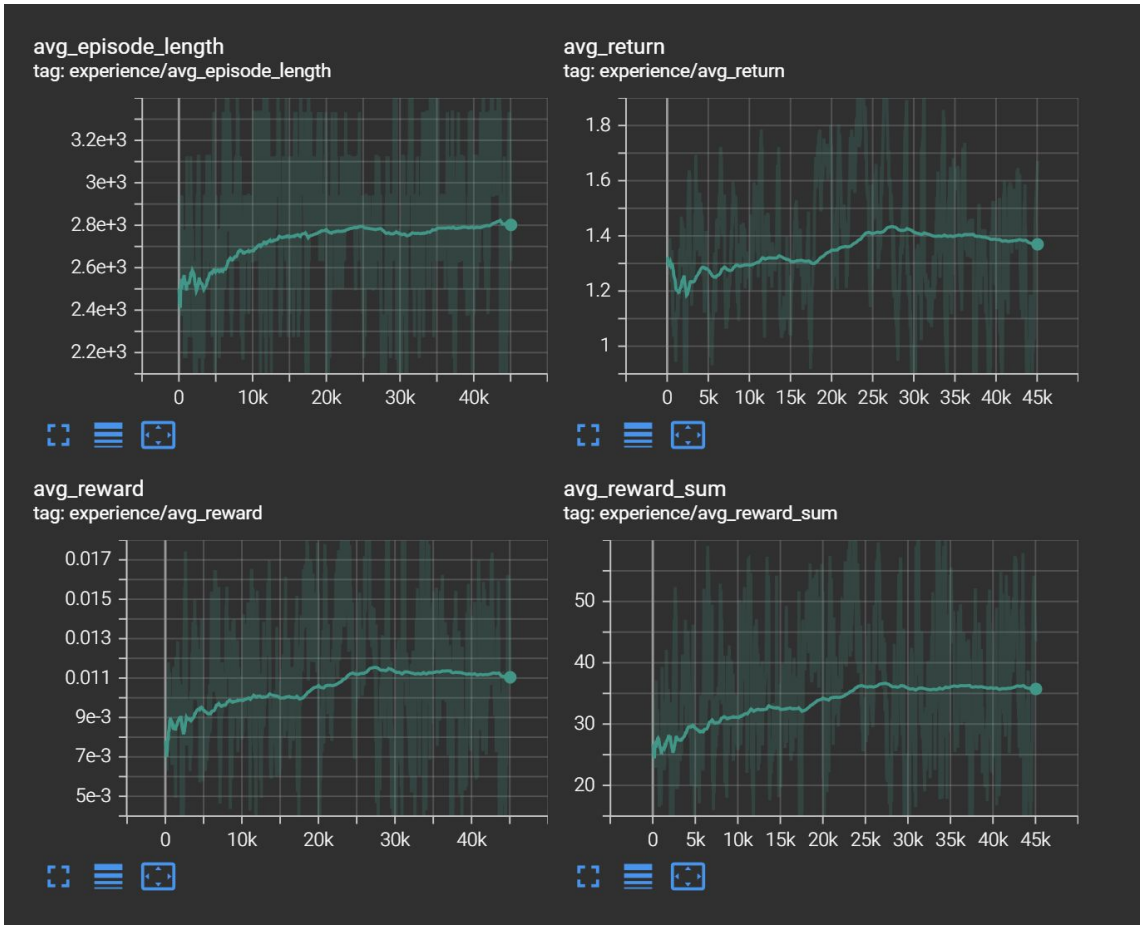


Figure 18: High Fidelity Simulation - Advanced Track Based Training Graphs

4.2.5 Advanced track based - Quantitative Analysis

This final phase was less successful than the previous one. As shown in Figure 18, learning was minimal and peaked around 30,000 steps across all metrics. However, the unsmoothed raw data (visible in translucent green behind the curve) shows a large amplitude, indicating that the training was highly unstable and fluctuated constantly without stabilizing. This instability was caused by the ever-shifting race track and the random obstacles. The wide variation in performance suggests that the agents struggled to learn obstacle avoidance; receiving a mix of excellent and poor results likely confused the learning process.

4.2.6 Advanced track based - Qualitative Analysis

While the agents perform quite efficiently in open areas, they still lack reactivity when approaching obstacles, which frequently leads to collisions. To fix this, the

agents would need more extensive training in this specific type of environment, and potentially a revised reward function to better guide their behavior.

4.3 Statistic comparison

Table 1: Performance metrics across different drone racing configurations

Metric	**Simple track (200 tries)**	**Advanced track (200 tries)**	**Champion racer on Advanced track**
Success rate	89%	79%	98%
Average time	36.77s	97.49s	25.5s
Median time	36.84s	94.56s	25s
Shortest time	29.98s	53.98s	23s
Longest time	41.37s	158.04s	28s
Average number of collision	0.52	430	0.02
Median number of collision	0	398	0
Lowest number of collision	0	6	0
Highest number of collision	6	1547	1

To conclude the results, a statistical study was made to compare the performance of the different training on its own training environment against a human pilot on the advanced track (cf Figure 18). The simple track training gave really good results : only 10 seconds behind the champion on average and only 5 on its best score. Its collision amount is also close to perfect (mostly null), however this is less relevant as it does not have any obstacles. Its success rate is also quite good : approximately 9 agents on 10 finish the race; but it also means that 1 agent out of 10 never go out of the track which still is a high percentage. On the other hand, the advanced track is lacking in every domain : on average, the agent finishes the track in 97 seconds which is almost 4 times longer than the human. This is explained by the amount of collision (on average 430) that slows down the drone. The instability observed on the Tensorboard results is also present here : the difference between the shortest and the longest time is 104 seconds which represents twice the shortest time and the difference in collisions is 1541. It shows that the agent can have some really good results : 54 seconds is only twice the average time of the champion and 6 collisions is almost a perfect score. However it is still not controlled by the agent itself and seems to depend on sheer luck.

4.4 Limitations

Despite the encouraging performance observed on nominal tracks, a rigorous evaluation of the system highlights several major technical bottlenecks:

- **Sensitivity to Environmental Edge Cases:** The model suffers from a lack of generalization when the density of physical obstacles exceeds a critical threshold. The agent struggles to arbitrate between short-term avoidance maneuvers and maintaining its long-term racing trajectory.
- **Noise and Instability of the Reward Signal:** The introduction of randomized tracks and obstacles induces an apparent non-stationarity for the agent. The sharp variations between high crossing rewards and severe collision penalties disrupt the gradient updates of the PPO policy, as evidenced by the high variance in the un-smoothed data.
- **Perceptual Latency:** While the raycasting system is highly effective in open spaces, it encounters critical limitations at high velocities within confined environments. This issue stems from a temporal optimization strategy where the raycast calculations are distributed across three frames, processing only one-third of the total vectors per frame. Consequently, the complete environmental update suffers from refresh latency, which prevents the system from maintaining sufficient reflexive reactivity at top speeds. In confined spaces, this lag creates a critical gap between perception and positioning, leading to inertia-driven collisions as the drone cannot react in time to updated environmental data.

5 Conclusions and Future Perspectives

The objective of this final year project was to design an autonomous artificial intelligence capable of piloting an FPV drone at high speeds within complex and realistic three-dimensional environments. To achieve this, we implemented a deep reinforcement learning pipeline based on the Proximal Policy Optimization (PPO) algorithm, integrated via Epic Games' Learning Agents plugin for Unreal Engine 5 into the high-fidelity simulator TRYP FPV. Our incremental methodology successfully guided the agent's progression from a simple two-dimensional navigation setup to a three-dimensional flight constrained by racing gates and obstacles.

Quantitative results demonstrate the viability of this approach on standard tracks. The trained agent achieves excellent performance, trailing a human champion's average time by only 10 seconds, with a 5-second gap on its best score, and maintaining a 90 percent race completion rate. However, evaluation on the advanced track highlights the current limitations of the model regarding generalization. When faced with a high density of obstacles, the instability observed in the learning curves and the high number of collisions (430 on average) reveal a clear lack of reactivity and anticipation from the AI during high-velocity flight phases.

Several development axes are envisioned to overcome these limitations:

- **Perception Optimization:** Increasing the density and range of the forward-facing raycast cone, or transitioning toward a direct computer vision approach (End-to-End learning from the FPV camera feed).
- **Refining Reward Shaping:** Adjusting the cost functions to penalize the critical approach to obstacles more severely, well before a physical collision occurs.
- **Network Architecture:** Integrating recurrent neural networks (RNN/LSTM) to provide the agent with a short-term temporal memory of its trajectory, which is essential for anticipating sharp turns and hidden obstacles.

Ultimately, this framework lays the foundation for advanced training tools within the video game industry (such as immersive racing bots for Undead Battery) as well as for evaluating flight policies applicable to real-world drone navigation.

References

[1] Undead Battery. Undead Battery's Website, 2012.

[2] Undead Battery. TRYP FPV: The Drone Racer Simulator, 2022.

[3] Amazon. Prime Air, 2015.

[4] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Mueller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. Nature, 2023.

[5] Jean Tampon and Alexis Louis. Training a neural network to operate drones using genetic algorithm. Pezzza's Work's YouTube channel, 2020.

[6] Alexandre Sajus. Controlling drones with ai (python reinforcement learning quadcopter). Alexandre Sajus's YouTube channel, 2023.

[7] Epic Games. Learning Agents, 2023.

[8] Rui Yu, Shenghua Wan, Yucen Wang, Chen-Xiao Gao, Le Gan, Zongzhang Zhang, and De-Chuan Zhan. Reward models in deep reinforcement learning: A survey. Nanjing University, 2025.

[9] Brendan Mulcahy, Daniel Holden, and Gin Wang. Learning agents (5.5). Epic Games (Unreal Engine website), 2024.